캡스톤 디자인

23101177 컴퓨터공학과 박재홍

Archery

목차

1. 배경 — 게임 목표

2. 배경 — 기술적 목표

3. 게임 시스템 소개

4. 개발 일정

배경 — 게임 목표

🎮 이런 게임을 만들고 싶었습니다

「궁수의 전설 2」 같은 **캐주얼 로그라이크 게임**을

직접 만들어보고 싶다는 목표에서 시작

게임 플레이는 모바일 캐주얼 게임을 목표로 만들었기 때문에
이동하면서 몬스터와 몬스터의 투사체를 피하고 정지하면
적에게 투사체를 던져서 공격하는 **쉬운 조작 방법**을 채택

플레이어는 스테이지 내에서 몬스터를 처치하고 얻은 경험치를
통해서 **레벨업**을 하게 되면 **스킬들을 조합**하여 플레이 진행
또한 스테이지를 해결하면서 얻은 **재화**를 통해서 플레이어는
새로운 캐릭터를 구매, 장비를 구매하여 새로운 맵을 클리어하는
게임

배경 — 기술적 목표

기존 방식의 문제

새로 빌드를 하지 않고 컨텐츠를 추가하는 시스템 구축이 목표

에셋 관리 방식 : SerializeField를 통한 직접 바인딩

일반적으로 Unity에서 적(Enemy)이나 플레이어(Player) 같은

프리팹을 사용하려면 **[SerializeField]**로 Inspector에

직접 드래그해서 연결

새로운 적 종류를 추가할 때마다
- 스크립트에 새 필드를 만들고
- 유니티 게임 설정창에서 일일이 다시 연결해야 하고
- 실수로 빠뜨리면 런타임 에러 발생!

게다가 새로운 콘텐츠를 넣기 위해서는 계속 다시 빌드를 해야함

→ 비용이 지속적으로 발생!

다른 대안들도 불편

에셋 관리 방식 : Resources 폴더 / Asset Bundle

Unity에는 기존에도 에셋을 불러오는 다른 방법이 있지만,

각각 단점이 있습니다.

Resources 폴더

· 문자열로 경로를 써서 로드 → 오타 시 에러
· Resources 안의 모든 파일이 빌드에 포함
· 안 쓰는 파일도 빌드에 포함되어 **앱 용량 증가**
· 공식적으로 Unity도 **"사용을 지양하라"**고 권고

Asset Bundle

· 번들 생성·관리·버전 관리를 전부 직접 구현
· 의존성 관리, 설정이 매우 **복잡**
· **코드량이 많아** 소규모 팀에 부담

배경 — Addressable Assets 도입

✅ 해결책: Addressable Assets

Unity의 **Addressable Assets**은 쉽게 말해 **이름표(라벨)를 붙여놓고, 필요할 때 이름표를 통해 불러 오는 시스템**

✨ 왜 도입하는가?

1. 콘텐츠 추가 간편

새로운 게임 오브젝트를 만들면 이를 addressable로 등록 후 라벨만 붙이면 자동으로 유니티 시스템이 분류하고 게임에 배치

코드 수정없이 바로 콘텐츠를 투입 가능

2. 메모리 효율적

필요한 순간에만 로드하고, 안 쓰면 쉽게 해제 가능.

메모리적으로 게임이 무거워지는 것을 방지

3. 라벨 기반 일괄 로드

“skill_config”과 같은 직접 설정한 라벨이 붙은 에셋을 한 번에 전부 로드

개별적으로 메모리 경로를 따로 코드에 기입할 필요 X

4. 목표

모바일 게임이나 게임 회사들이 **새로운 빌드 없이 새로운 컨텐츠를 추가**하는 방식의 게임 시스템을 구축하는 것이

이번 프로젝트의 기술적 목표

게임 시스템 ① — 전체 게임 흐름

1. **로딩 화면** — 게임 시작 시 모든 데이터(적 정보, 맵 정보, 스킬 정보)를
미리 불러와 준비합니다. 로딩 진행률이 화면에 표시됩니다.

2. **로비 화면** — 플레이할 **캐릭터와 장비**를 세팅하고
도전할 **맵**을 선택합니다.

3. **인게임 (전투)** — 스테이지별로 몰려오는 적을 물리치고,
레벨업 시 **스킬을 선택**하여 플레이어를 점점 강화해
나갑니다.

4. **스테이지 클리어** — 모든 적을 처치하면 포탈이
나타나고,
포탈에 진입하면 다음 스테이지로 이동합니다.

5. **맵 클리어 또는 사망** — 모든 스테이지를 클리어하면 로비로
복귀.
사망 시 **재도전** 또는 **로비 복귀**를 선택할 수 있습니다.

게임 시스템 ② — 로비 · 캐릭터 선택

로비 화면 – 캐릭터 선택

게임 시작 전 캐릭터를 선택하는 로비 화면입니다.

- **캐릭터 미리보기**
  - ❖ 로비 화면에 선택된 캐릭터의 3D 모델로 미리보기 표시
  - ❖ 좌/우 버튼으로 캐릭터를 전환하고, 선택 버튼으로 확정
  - ❖ 구매하지 못하여 아직 잠금된 캐릭터의 경우 잠금 아이콘 표시

- **캐릭터 구매**
  - ❖ 골드를 소비하여 새로운 캐릭터를 구매
  - ❖ 캐릭터마다 고유 스탯(HP, 공격력, 이동속도 등)이 다르게 설정
  - ❖ 캐릭터마다 고유 레벨을 가지고 있어 기존 캐릭터를 레벨업 시켜서 계속 육성 가능

게임 시스템 ② — 로비 · 장비 선택

🏠 로비 화면 – 장비 선택

게임 시작 전 장비를 선택하는 로비 화면

- **장비 구매**

❖ 무기, 보호구, 신발 3종류로 구성
❖ 골드를 소비하여 새로운 장비를 구매 가능

- **장비 스텟**

❖ 장비는 각 종류마다 레벨을 공유
❖ 장비는 골드 소비를 통해서 레벨업 가능
❖ 레벨업 시킬 때마다 해당 종류의 장비들이 모두 스탯이 증가.
❖ 교체하려고 하는 장비와 착용 중인 장비의 스탯을 비교할 수 있도록 UI로 표시

게임 시스템 ② — 로비 · 맵 선택

로비 화면 – 맵 선택

게임을 시작하기 위한 맵 선택 화면

- **맵 미리보기**
  - ❖ 선택 가능한 맵을 3D 모델로 미리보기 표시
  - ❖ 위/아래 키로 맵을 전환
- **맵 잠금**
  - ❖ 이전 맵을 클리어해야 다음 맵이 해금
  - ❖ 해금되지 않은 상위 맵은 잠금 아이콘으로 표시
- **맵 구성**
  - ❖ 맵은 공유된 컨셉(던전, 정글 등)의 몇가지 맵 오브젝트로 구성
  - ❖ 맵 마다 나오는 몬스터의 구성이 다르도록 설정

게임 시스템 ③ — 이동 · 공격 ·피격

🏃 이동

- 키보드 방향키로 플레이어를 이동
- 이동 중에는 공격이 중지되고, 정지하면 자동으로 공격을 재개

⚔️ 자동 전투

- 플레이어는 가장 가까운 적을 **자동으로 탐지**하여 투사체를 발사
- 공격 속도, 투사체 수, 투사체 방향은 스킬과 장비에 의해 변동

💀 피격 · 사망

- 적에게 닿거나 투사체에 닿으면 데미지를 받습니다
- 체력이 0이 되면 **게임오버 팝업**이 표시
- 게임 오버 팝업에서 로비 화면으로 가거나 재도전을 선택 가능

🔥 마법 속성

- **화염(Fire)** — 일정 시간 동안 지속 데미지
- **독(Poison)** — 느린 지속 데미지
- **빙결(Ice)** — 이동 속도 감소
- **번개(Lightning)** — 감전
- **마그마(Magma)** — 데미지 증폭

게임 시스템 ④ — 스킬 · 레벨업 시스템

📈 레벨업

- 적을 처치하면 **경험치 보석**이 드롭
- 경험치가 가득 차면 레벨업하며, 게임이 **일시정지**
- 3개의 스킬 중 **1개를 선택**하여 캐릭터를 강화

🎯 스킬

- **플레이어 스탯 강화 스킬**
  - ➢ 공격속도 증가 — 투사체 발사 간격 감소
  - ➢ 이동속도 증가 — 캐릭터 이동속도 증가
- **투사체 강화 스킬**
  - ➢ 멀티샷 — 기본 공격 화살 수 증가
  - ➢ 대각선샷 — 좌우 대각선 방향 추가 투사체
  - ➢ 수평샷 — 좌우 수평 방향 추가 투사체
- **공격 유형 추가 스킬**
  - ➢ 배럴 — 배럴이 주기적으로 생성되고 배럴에 접촉 시 공중 폭격 생성
  - ➢ 오브 — 캐릭터 주위를 도는 회전체 생성

게임 시스템 ⑤ — 적 시스템

적의 태그 시스템 (Bit Flag)

적은 **비트 플래그 기반 태그**를 조합하여 정의

유형 태그 (Identity)

태그	설명
Melee	근접형 적
Ranged	원거리형 적
Boss	보스 — 높은 체력, 특수 패턴

이동 모듈 (Move)

모듈	설명
FollowMove	플레이어를 향해 추적 이동
PatternMove	정해진 경로를 순회하며 이동
RandomMove	랜덤 방향으로 이동

예: Ranged + RandomMove + Shoot + Fire → 랜덤 방향 이동하는 화염 원거리 몬스터

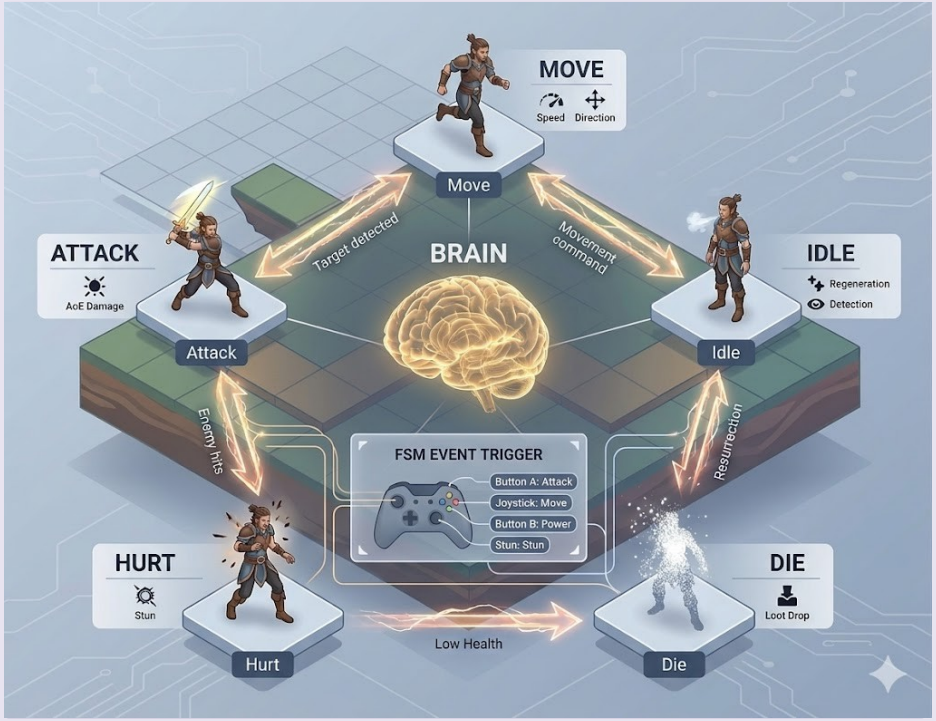
공격 모듈 (Attack)

모듈	설명
MeleeAttack	근접 타격 공격
FollowMeleeAttack	추적 후 근접 공격 (돌진형)
Shoot	투사체 발사 (원거리)
FlyingShoot	공중 투사체 발사

속성 태그 (Attribute)

속성	효과
Fire	화염 — 지속 데미지
Ice	빙결 — 이동속도 감소
Poison	독 — 느린 지속 데미지
Lightning	번개 – 감전
Magma	마그마 – 데미지 증폭
Normal	속성 없음

게임 시스템 ⑤ — Enemy FSM (상태 머신)



적 상태 머신 (FSM)

각 적은 **Idle · Move · Attack · Hurt · Die**

5가지 상태를 가지며, 조건에 따라 전이

Enemy Brain

상태는 조건에 따라 전이하여 다른 상태로 전환

전이 이후의 상태를 **Enemy Brain** 이라는

Scriptable Object(유니티 데이터 컨테이너)에서 정의하여 전이 패턴을 관리

각 적마다 **서로 다른 EnemyBrain SO**를 할당하여 상태 흐름을 다르게 설정할 수 있습니다.

예시:

· 근접 적: Idle → **Move** → Attack

· 원거리 적: Idle → **Attack** (이동 없이 즉시 공격)

· 보스: Idle → **Move** → Attack (특수 패턴)

게임 시스템 ⑥ — 스테이지 · 맵 시스템

🔄 스테이지 진행 사이클

1. **전투 시작** — 적들이 스폰되어 플레이어를 공격
2. **적 전멸** — 모든 적을 처치하면 클리어
3. **포탈 등장** — 포탈에 진입하여 다음 스테이지로 이동
4. **다음 스테이지 준비** — 새로운 맵·적 로드 후 반복
5. **맵 클리어!** — 모든 스테이지 완료 시 로비 복귀

🗺️ 스테이지 내 맵 구성

- **기본 맵** : 맵에서 나오기로 설정된 종류의 적들이 랜덤 생성
- **스폰 지정 맵** : 해당 맵에서는 특정 Enemy가 미리 설정되어 있는 위치에서 생성
- **보스 맵** : 10스테이지 주기로 보스 Enemy가 생성

⚙️ 맵 시스템 설계

- 수치 데이터(스탯)는 **JSON**
- 에셋 참조(Enemy 게임 오브젝트)는 **Scriptable Object(SO)**
- 두 데이터는 **mapId** 라는 키로 연결되어 사용
- 기본 맵과 스폰 지정 맵은 SO에 등록되어 랜덤 생성
- 보스 맵은 SO에서 따로 등록되어 생성
- 상위 스테이지로 갈수록 적의 스탯이 항상되어 난이도가 상승

게임 시스템 ⑦ — 스탯 시스템

🧙플레이어 3계층 스탯 구조

- **기본(Base)** — 캐릭터 고유 능력치
- **장비(Equipment)** — 장비 착용 시 얻어지는 스탯 증가량
- **인게임 버프(Buff)** — 레벨업 스킬 선택 시 해당 판 한정 스탯 효과

📊 스탯 계산

- **최종 스탯** = 캐릭터 스탯 + 장비 스탯 + 인게임 버프
- **인게임 버프** — 맵 클리어 또는 사망 시 초기화됩니다

👾 Enemy 스탯 구조

- **기본(Base)** —적의 고유 능력치
- **맵 배율(Map Modifier)** — 맵마다 적용되는 맵의 고유한 스탯 배율. 상위 맵일수록 맵 배율 증가하게 되어 난이도 상승.
- **스테이지 배율 (Stage Growth)**—스테이지마다 적용되는 스탯 배율. 스테이지가 진행될수록 스탯이 증가되어 난이도 상승.

📊 스탯 계산

- **최종 스탯** = 기본 스탯 × 맵 배율 × 스테이지 배율
- 후반 맵의 높은 스테이지일수록 자연스럽게 **난이도가 상승**

개발 일정

졸업 작품 개발 일정

주요내용	추진일정												기간
	3월				4월				5월				
계획서 발표													1주
로비 시스템 개발													12주
맵 시스템 개발													4주
캐릭터 로비 시스템 개발													4주
1차 발표													1주
장비 로비 시스템 개발													4주
2차 발표													1주
시현													1주

감사합니다